SMART EMERGENCY ESCAPE PLANNER

A GUI-Based Shortest-Path Application Using Dijkstra's Algorithm

Student Name	Jayanth Mathew
Roll Number	14
Subject	Design and Analysis of Algorithms (DAA)
Project Type	Graph algorithm visualisation
Technology	Python, Tkinter, NetworkX, Matplotlib

Prepared for academic submission. The building layout and corridor costs are simulated for educational demonstration; this is not a certified emergency-navigation system.

1. Problem Statement

During an emergency, a person may need to find a low-cost route from their current location to a building exit. A corridor may become unavailable, making the original route unsuitable. This project models a building as a weighted graph and uses Dijkstra's shortest-path algorithm to find a minimum-cost route to a selected exit or the nearest reachable exit. Users can block or reopen corridors and observe how the route changes.

2. Objectives

• Represent rooms, halls, staircases, and exits as graph vertices.
• Represent corridors as weighted edges, with weights representing simulated travel cost.
• Calculate a minimum-cost route using Dijkstra's algorithm.
• Allow corridors to be blocked and reopened, then recalculate the route.
• Display the graph, highlighted route, total cost, and algorithm steps in a GUI.

3. Tools and Technologies

Python: application logic and Dijkstra implementation.
Tkinter: graphical user interface and controls.
NetworkX: graph representation and drawing support.
Matplotlib: embedded building graph visualisation.
heapq: priority queue used by Dijkstra's algorithm.

4. Graph Model

Each room, hall, staircase, and exit is a vertex (V). Each corridor is an edge (E). Every edge has a non-negative weight representing a simulated cost. The application removes an edge when its corridor is blocked and restores the edge when reopened. After a change, the shortest-path calculation is run again.

5. Algorithm: Dijkstra's Shortest Path

Input: Weighted building graph, starting location, and destination preference.
Output: Minimum-cost route and total cost, or a message when the selected exit is unreachable.

Algorithm steps

1. Set the starting vertex distance to 0 and all other distances to infinity.
2. Insert the starting vertex into a min-priority queue.
3. Remove the vertex with the smallest known distance.
4. For each adjacent vertex, calculate the new distance through the current vertex.
5. If the new distance is smaller, update the distance and predecessor, then add the updated entry to the queue.
6. Continue until all reachable vertices have been processed.
7. Choose the requested exit, or the reachable exit with the smallest total cost.
8. Reconstruct the route by following predecessor links.
9. If a corridor changes state, update the graph and repeat the calculation.

Pseudocode

```
DIJKSTRA(graph, start):
for each vertex v: dist[v] = infinity; prev[v] = null
dist[start] = 0
priority_queue = [(0, start)]
while priority_queue is not empty:
(d, u) = extract_min(priority_queue)
if d != dist[u]: continue
for each neighbor v of u with weight w:
if dist[u] + w < dist[v]:
dist[v] = dist[u] + w
prev[v] = u
insert (dist[v], v) into priority_queue
return dist, prev
```

6. Complexity Analysis

Let V be the number of vertices (locations) and E be the number of edges (corridors). With an adjacency-list graph representation and a binary heap priority queue, Dijkstra's algorithm has the following bounds.

Measure	Complexity	Explanation
Time	O((V + E) log V)	Priority queue operations and edge relaxations.
Space	O(V + E)	Graph storage, distances, predecessors, and queue.

Dijkstra's algorithm requires non-negative edge weights.

7. GUI Screenshots

The screenshots below show the normal route, a blocked-corridor alternative, the restored route after reopening the corridor, and a route from another starting location.

Figure 1 — Normal shortest route

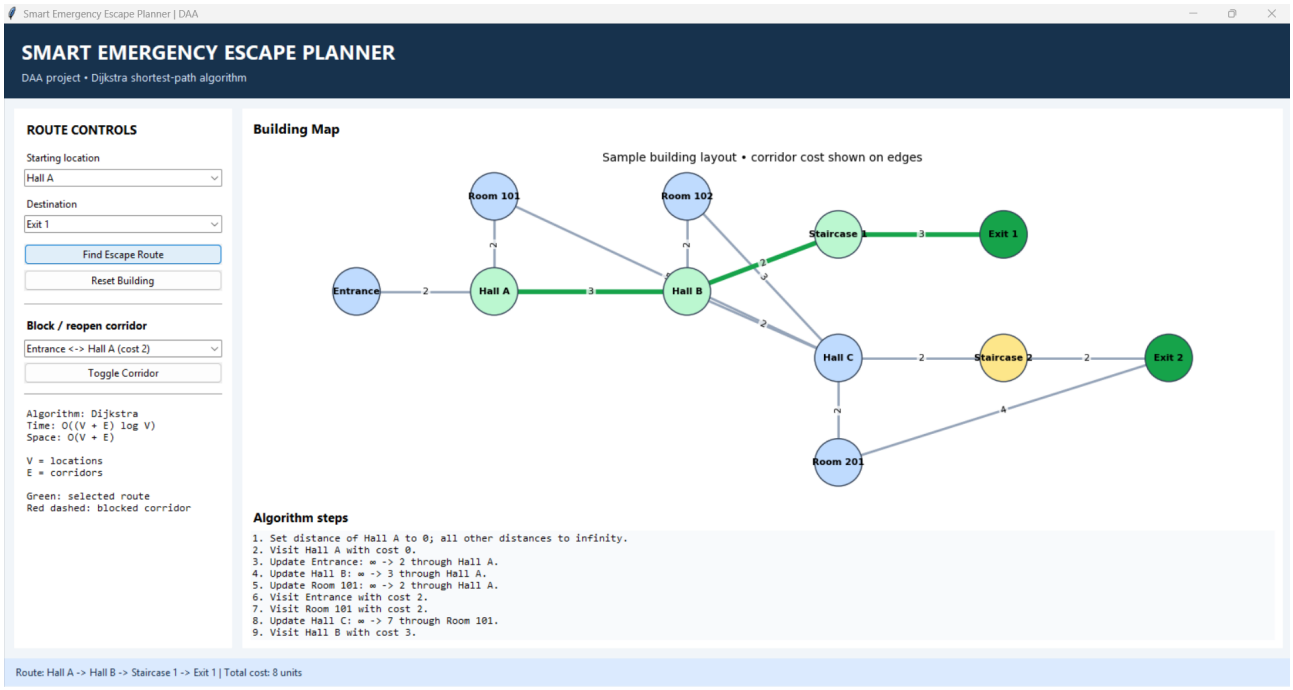


Figure 1. Hall A to Exit 1: Hall A → Hall B → Staircase 1 → Exit 1; total cost 8 units.

Figure 2 — Corridor blocked

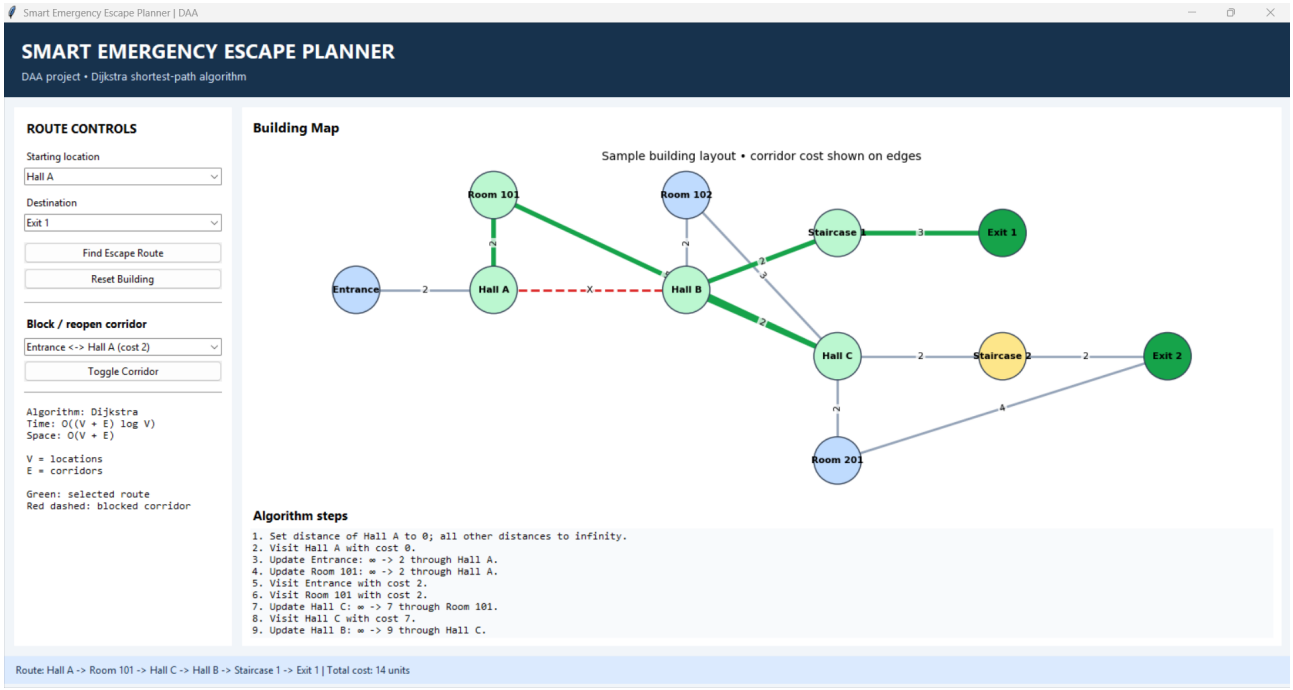


Figure 2. Hall A–Hall B corridor blocked; alternative route via Room 101 and Hall C; total cost 14 units.

Figure 3 — Corridor reopened

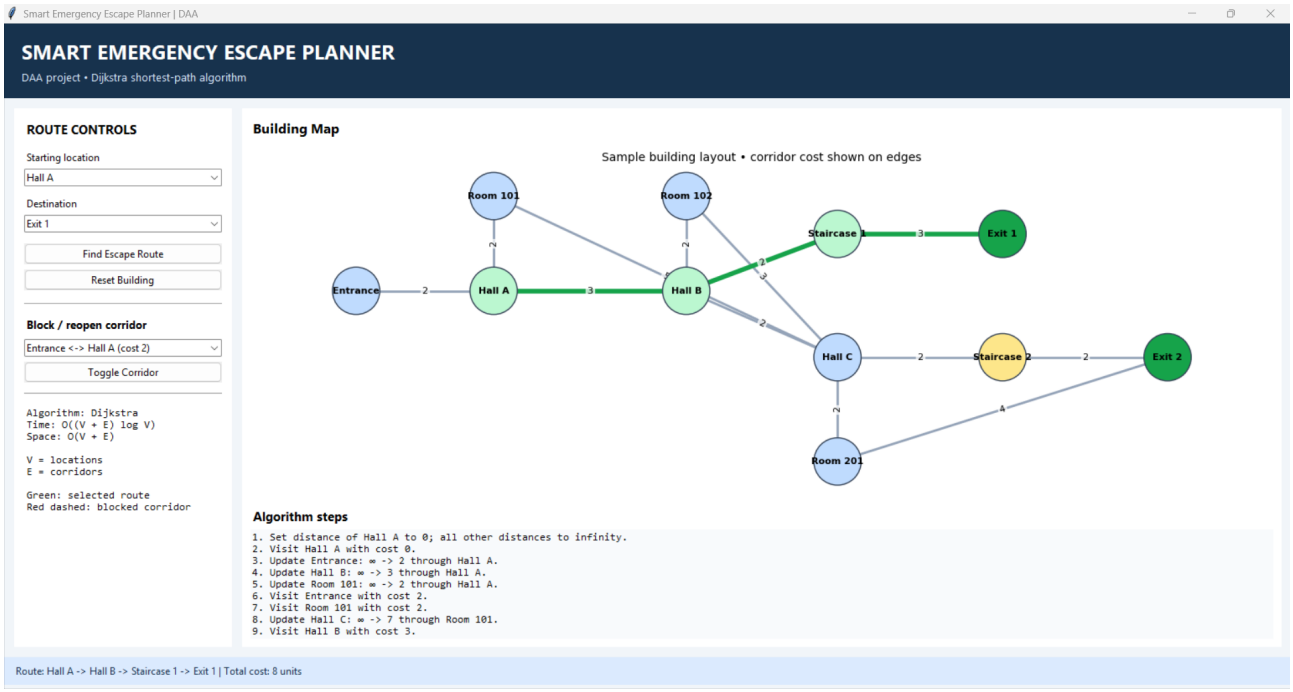


Figure 3. After reopening Hall A–Hall B, the lower-cost route is restored; total cost 8 units.

Figure 4 — Different starting location

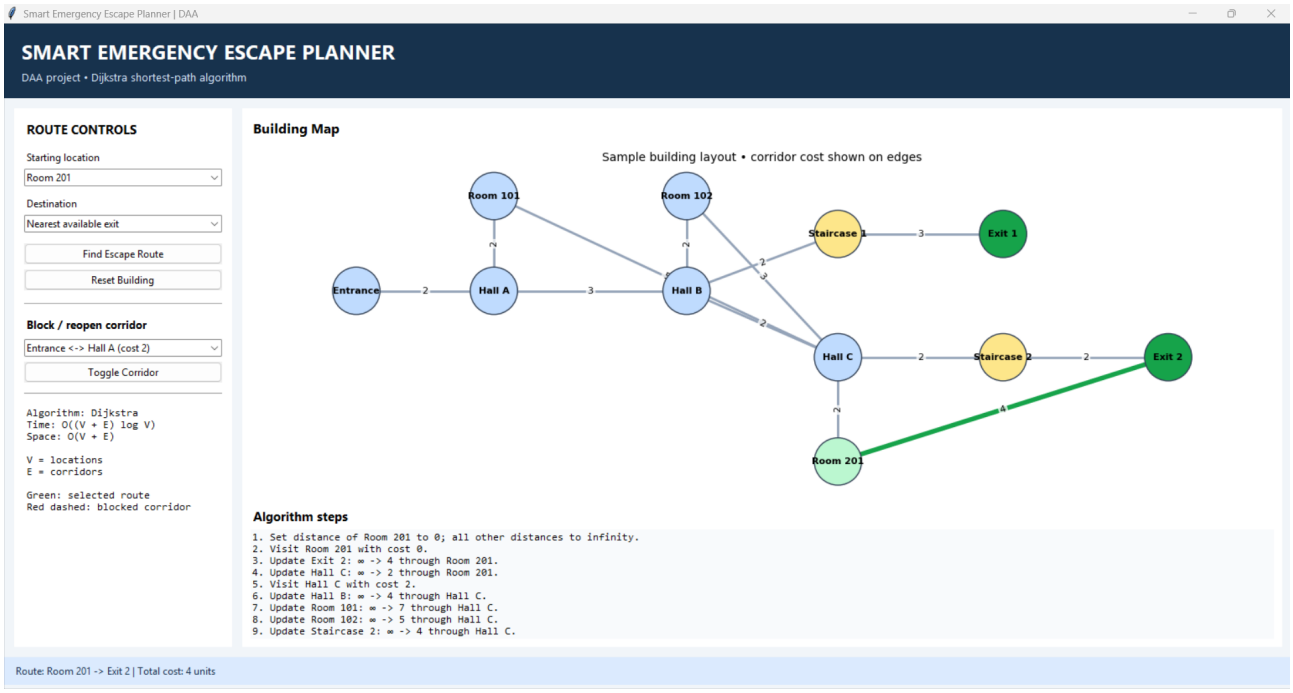


Figure 4. From Room 201, the nearest exit is Exit 2 via a direct connection; total cost 4 units.

8. Test Results

The following results were observed in the application screenshots.

Test	Scenario	Observed result	Status
1	Hall A to Exit 1, corridors open	Hall A → Hall B → Staircase 1 → Exit 1; cost 8	Pass
2	Block Hall A–Hall B	Alternative via Room 101 and Hall C; cost 14	Pass
3	Reopen Hall A–Hall B	Lower-cost route restored; cost 8	Pass
4	Room 201 to nearest exit	Room 201 → Exit 2; cost 4	Pass
5	Unreachable exit	Not captured/tested for this report	Not tested

9. Conclusion

The Smart Emergency Escape Planner demonstrates Dijkstra's shortest-path algorithm on a weighted graph through an interactive GUI. The screenshots show that the application finds a minimum-cost route, recalculates an alternative route after a corridor is blocked, restores the lower-cost route when a corridor is reopened, and works from a different starting location. The building and costs are simulated for educational purposes; this is not a certified real-world emergency system.